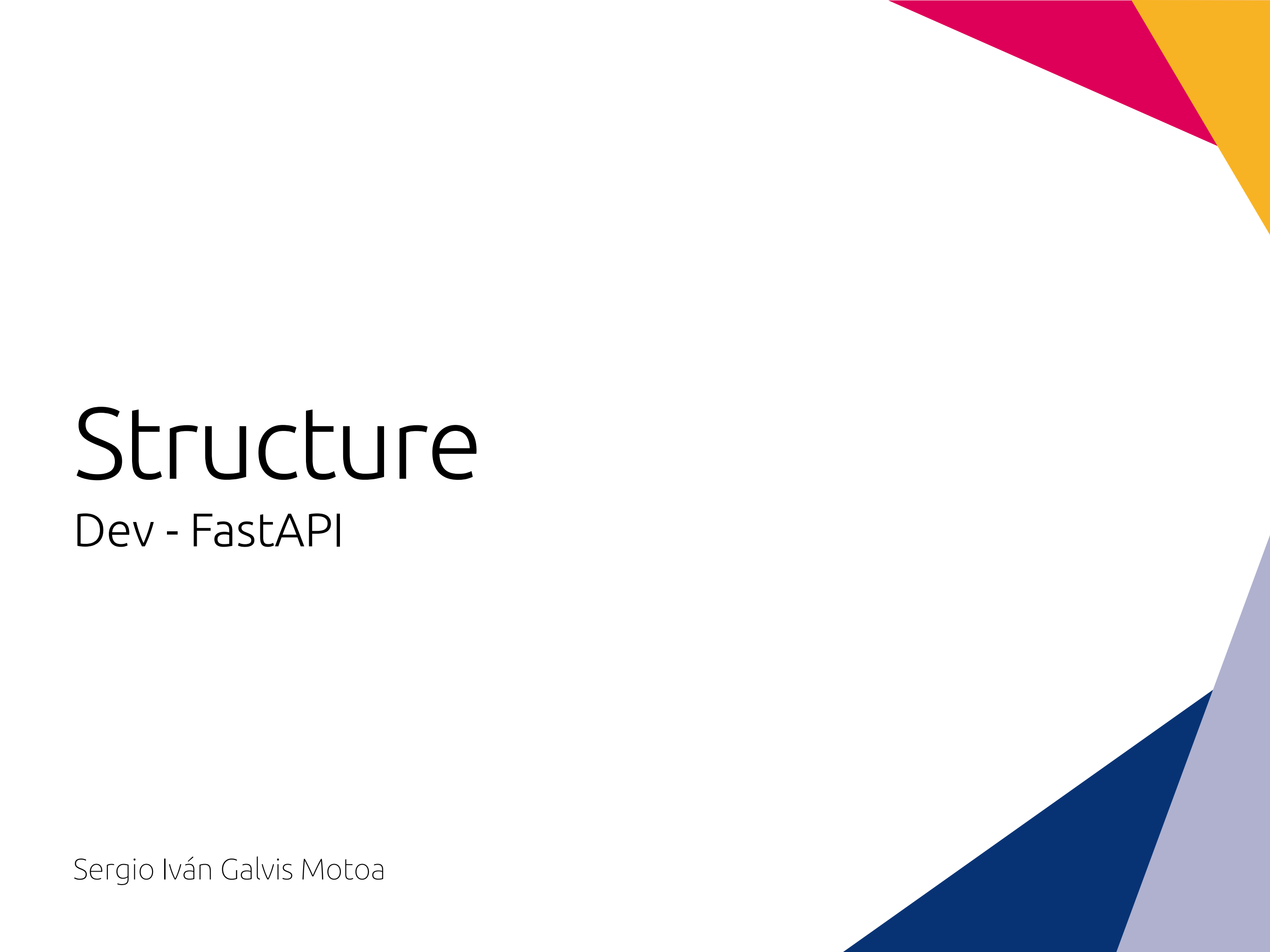


The slide features decorative geometric shapes in the corners. The top right corner has overlapping magenta and yellow triangles. The bottom right corner has overlapping dark blue and light blue triangles.

Structure

Dev - FastAPI

Sergio Iván Galvis Motoa

Structure

- At this point (after the first F.U.C.K) the project looks well, but is so important to identify improve opportunities, in Software it happens always, you never will have the perfect and final solution, always are allowed to improve.
- The best practice (preferred from the beginning of a project) is define a good structure for the project. Depending in the philosophy or methodology adopted you should to organize the project. SOLID is widely recommended for many authors.

* Don't hesitate to look for many info about SOLID principles and Clean Code.

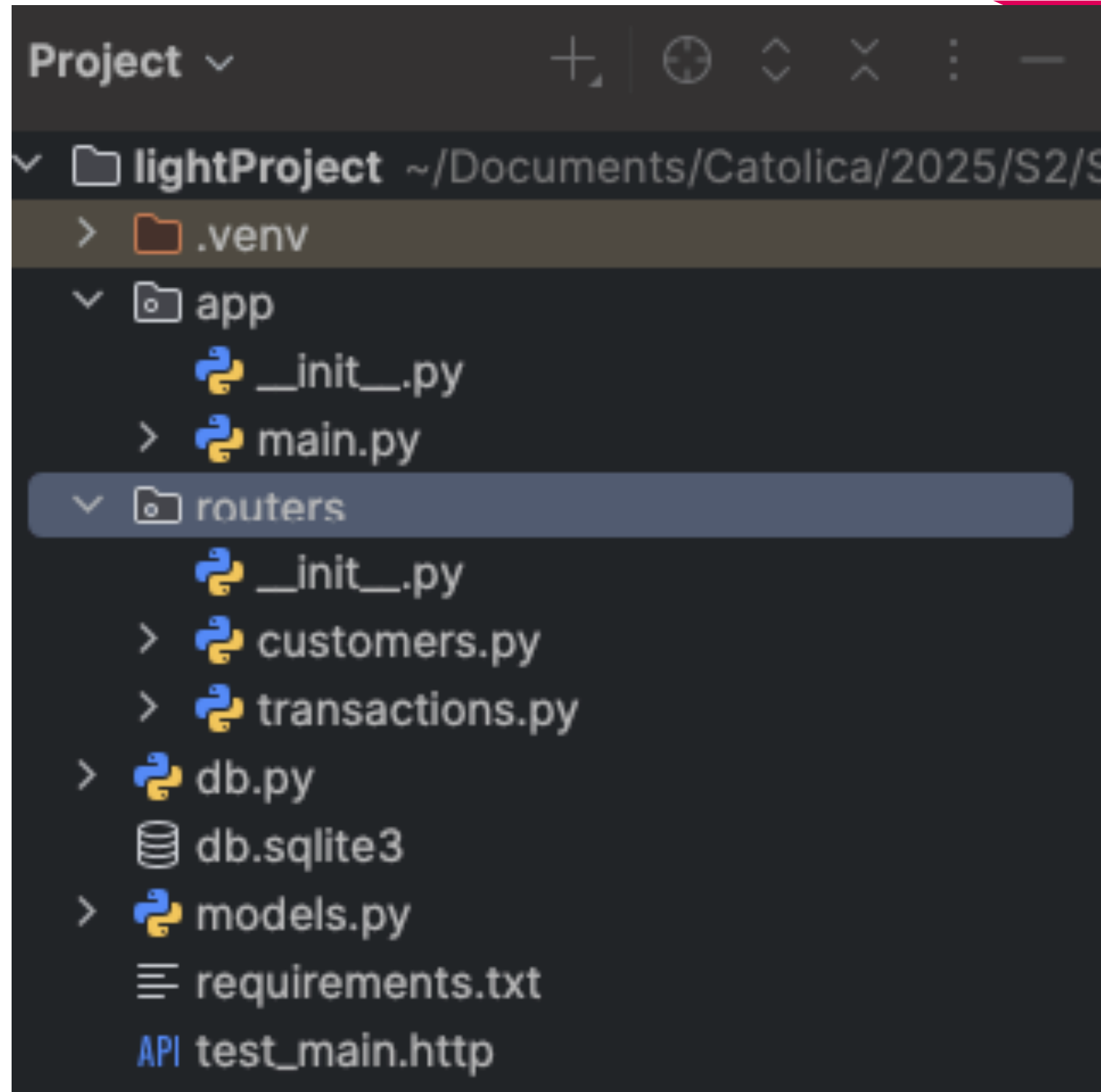
Structure

Planning

At this moment all the answers could be: “Depends”. The size, the scaling, the performance and many others. This structure and good practices has the goal of save time, effort and life in the future of the project.

- **Models:** Should be separated of the logic and functions. Depends the quantity of models you could have a file for each model (this idea doesn't have any rule like more than 3 models or a number of)
- **Routers:** This concept allow to identify each model interaction file (i.e. the F.U.C.K. of each one) allowing you to improve or fix without illness, like a separation of responsibilities. Some documentation treat routers like a mini-apps.
- **Main:** As always will be the entry point and will manage the “mini-apps” of the routers. With good practices could create a nice documentation from it.

*When you need to add a directory with files .py you need to add a “__init__.py” file, it'll allows you think in this file like a module importable in other files/modules



Project Tree

Structure

Structure

Router

- Is another file (with the code) obtained from the main.
- Now the instance of FastAPI is APIRouter (mini-app) and is called "router".
- The new path decorators will use the new instance: "router"
- This file will be called from main.py to be included in the main app instance.
- The tag in the instance "router" allows to group all the endpoints of this router. It means better documentation.
- The rest of the F.U.C.K structure should be develop in the file.

```
customers.py

from fastapi import APIRouter, HTTPException
from sqlmodel import select
from db import SessionDep
from models import Customer, CustomerCreate, CustomerUpdate

# /
router = APIRouter(tags=["customers"])

# /costumer
@router.post(path: "/costumer", response_model=Customer, status_code=201)
def create_costumer(
    costumer: CustomerCreate, session: SessionDep
):
    db_costumer = Customer.model_validate(costumer)
    session.add(db_costumer)
    session.commit()
    session.refresh(db_costumer)
    return db_costumer
```

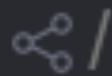
Structure

Main

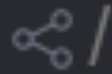
- More clean in the imports.
- Just include the router, and the app will add the endpoints in the documentation and the service.

main.py

```
from routers import customers
from fastapi import FastAPI
from db import create_tables
```



```
app = FastAPI(lifespan=create_tables)
```



```
app.include_router(customers.router)
```